

IT4079:NGÔN NGỮ' và PHƯƠNG PHÁP DỊCH

Phạm Đăng Hải

haipd@soict.hust.edu.vn

Chương 4: Phân tích ngữ nghĩa

1. Giới thiệu
2. Bảng ký hiệu
3. Chương trình dịch định hướng cú pháp
4. Kiểm tra kiểu
5. Xử lý sai sót

Ví dụ 1

Cho văn phạm $G = (V_T, V_N, P, S)$

$P: \{ \langle \text{Câu} \rangle \rightarrow \langle \text{Chủ ngữ} \rangle \langle \text{Vị ngữ} \rangle$

$\langle \text{Chủ ngữ} \rangle \rightarrow \langle \text{Danh ngữ} \rangle | \langle \text{Danh từ} \rangle$

$\langle \text{Chủ ngữ} \rangle \rightarrow \langle \text{Danh ngữ} \rangle | \langle \text{Danh từ} \rangle$

$\langle \text{Danh ngữ} \rangle \rightarrow \langle \text{Danh từ} \rangle \langle \text{Tính từ} \rangle$

$\langle \text{Vị ngữ} \rangle \rightarrow \langle \text{Động từ} \rangle | \langle \text{Động từ} \rangle \langle \text{Bổ ngữ} \rangle$

$\langle \text{Bổ ngữ} \rangle \rightarrow \langle \text{Danh ngữ} \rangle$

$\langle \text{Danh từ} \rangle \rightarrow \langle \text{Bò} \rangle | \langle \text{Cỏ} \rangle$

$\langle \text{Tính từ} \rangle \rightarrow \langle \text{Vàng} \rangle | \langle \text{Non} \rangle$

$\langle \text{Động từ} \rangle \rightarrow \langle \text{gặm} \rangle$

Ví dụ 1

$L(G) =$

« Bò vàng gặm cỏ non »
« Bò vàng gặm cỏ vàng »
« Bò non gặm cỏ non »
« Bò vàng gặm bò non »
« Cỏ non gặm bò vàng »
.....

Các câu đều đúng ngữ pháp, nhưng không phải câu nào cũng đúng ngữ nghĩa (có ý nghĩa)

Ví dụ 2

```

Program Toto;
Const N = 0;
Begin
    N := 10;
End.
  
```

<Statement>

⇒ <Variable> := <Expression>

⇒ <Variableidentifier>:=<Expression>

⇒ N:= <Expression>

⇒ N:=<Term>

⇒ N:=<Factor>

⇒ N:=<Unsignedconstant>

⇒ N:=<unsignedinteger>

⇒ N:=10

Hoàn toàn đúng cú pháp của KPL

Sử dụng sai ý nghĩa ban đầu (Hằng số)

Nhận xét

- Không phải mọi câu văn (**NNLT**: câu lệnh) đúng ngữ pháp (**NNLT**: cú pháp) đều có giá trị sử dụng (**NNLT**: thực hiện được)
- Bộ phân tích ngữ nghĩa nhằm mục đích kiểm tra tính đúng đắn về mặt ngữ nghĩa của câu văn (**NNLT**: câu lệnh)

Nhiệm vụ bộ phân tích ngữ nghĩa trong NNLT

- Quản lý thông tin về các định danh (tên)
 - Hằng, biến, kiểu tự định nghĩa, chương trình con
- Kiểm tra việc sử dụng các định danh
 - Phải được khai báo trước khi dùng
 - Phải được sử dụng đúng mục đích
 - Gán giá trị cho hằng, tính toán trên kiểu, thủ tục...
 - Đảm bảo tính nhất quán
 - Tên được khai báo chỉ một lần trong phạm vi
 - Các phần tử trong kiểu liệt kê (*enum*) là duy nhất

Bảng ký hiệu

Nhiệm vụ bộ phân tích ngữ nghĩa trong NNLT

- Kiểm tra kiểu dữ liệu cho toán tử
 - Toán tử % của C đòi hỏi toán hạng kiểu nguyên
 - Có thể yêu cầu chuyển kiểu bắt buộc (*int2real*)
 - Chỉ số của mảng phải nguyên
- Kiểm tra sự tương ứng giữa tham số thực sự và hình thức
 - Số lượng tham số, tương ứng kiểu
- Kiểm tra kiểu trả về của hàm..

Các biểu thức kiểu của ngôn ngữ
Bộ luật để định kiểu cho các cấu trúc

Chương 4: Phân tích ngữ nghĩa

1. Giới thiệu

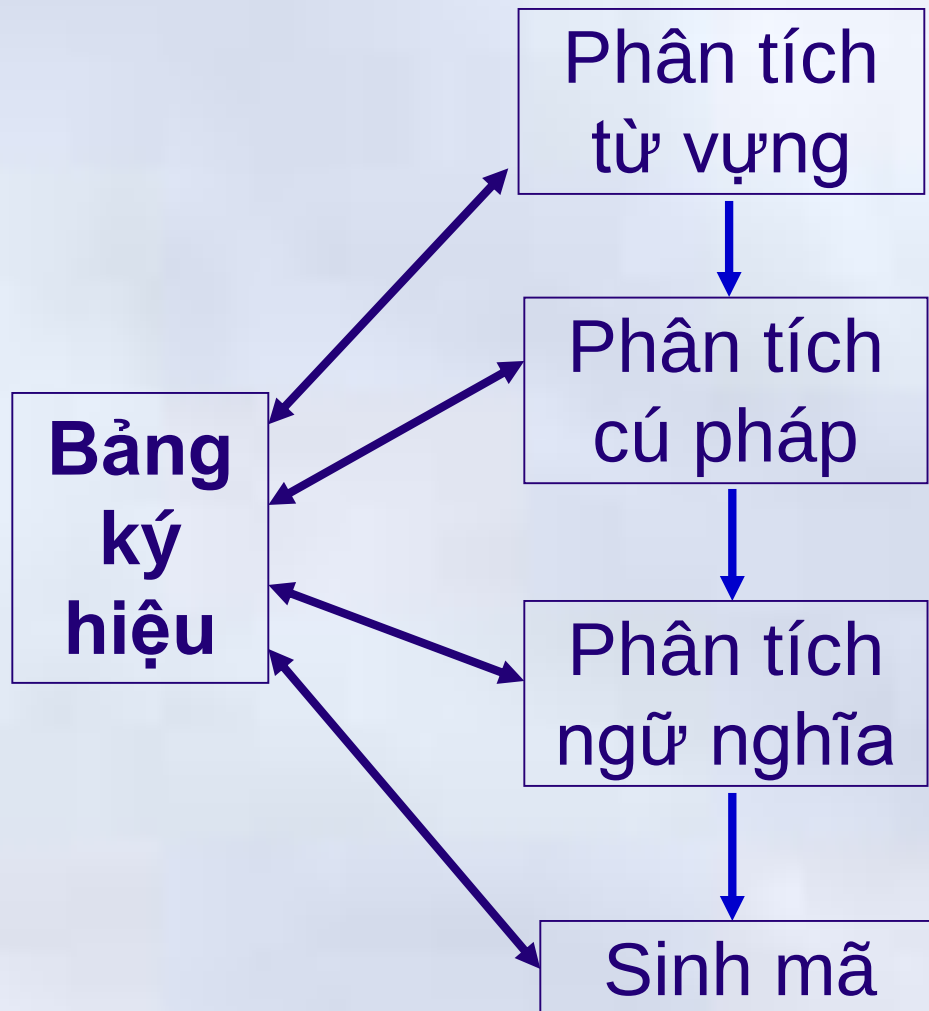
2. Bảng ký hiệu

3. Chương trình dịch định hướng cú pháp

4. Kiểm tra kiểu

5. Xử lý sai sót

Mục đích



- Bảng dữ liệu mà các pha của CTD đều s/dụng
- Dùng chứa thông tin về các danh biểu (**tên**) xuất hiện trong chương trình nguồn
- Mỗi phần tử ứng với một tên, gồm 2 trường
 - Trường tên
 - Khóa của bảng
 - Trường thuộc tính
 - Thuộc tính của tên
 - Biến (kiểu), hằng (giá trị)..

Mục đích

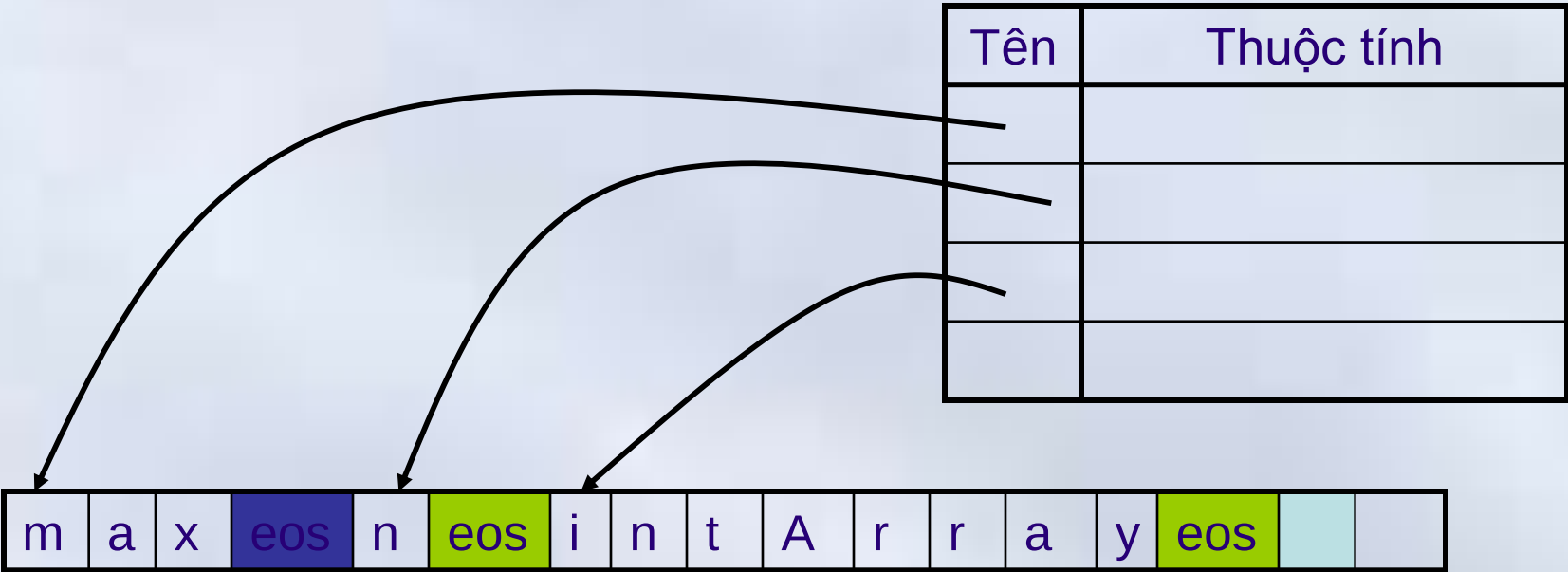
Khi gặp một danh biểu trong chương trình

- Gặp trong giai đoạn khai báo
 - Đưa tên và các thông tin tương ứng vào bảng
 - Ví dụ: **Const Max = 10;**
 - Đưa **Max** vào bảng, với kiểu là **constant**, giá trị là **10**;
- Gặp trong câu lệnh
 - Đọc thông tin ra để sử dụng
 - Phân tích ngữ nghĩa: Sử dụng đúng mục đích không?
 - Ví dụ: `Max := 20;` ← Sai mục đích
 - Sinh mã: Kích thước bộ nhớ cấp phát cho tên
 - Ví dụ: `int` → 2 bytes, `float` → 4 byte

Lưu trữ tên

Tên								Thuộc tính
m	a	x						
n								
i	n	t	A	r	r	a	y	

- Đơn giản
- Nhanh
- Độ dài tên bị giới hạn
- Lãng phí nhớ (~20%)



Hiệu quả hơn

Các yêu cầu phải có của bảng ký hiệu

- Phát hiện một tên cho trước có trong bảng hay không
- Thêm một tên vào bảng
- Lấy thuộc tính tương ứng với một tên
- Thêm các thông tin mới vào một tên
- Xóa một tên, nhóm tên ra khỏi bảng

Các cấu trúc dữ liệu cho bảng ký hiệu

Nhiều cách tổ chức bảng ký hiệu khác nhau

- Danh sách tuyến tính
 - Đơn gian, chậm
- Bảng băm
 - Nhanh, phức tạp
- Cây nhị phân tìm kiếm
 - Mức độ phức tạp và tốc độ vừa phải

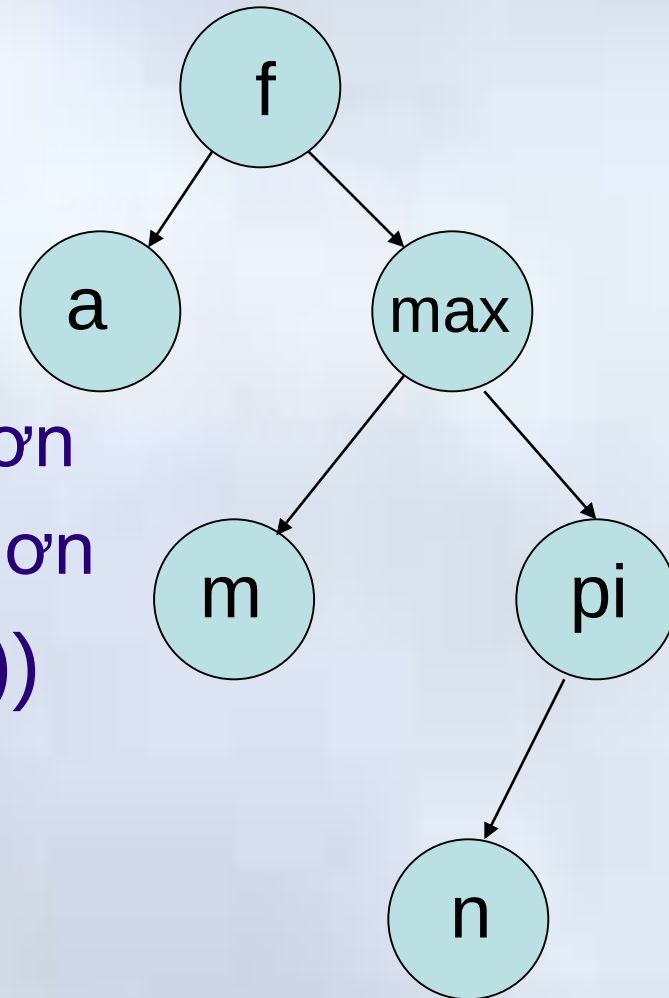
Bảng ký hiệu ảnh hưởng tới chất lượng của chương trình dịch vì CTD làm việc liên tục với bảng ký hiệu

Các CTDL cho bảng ký hiệu → Danh sách

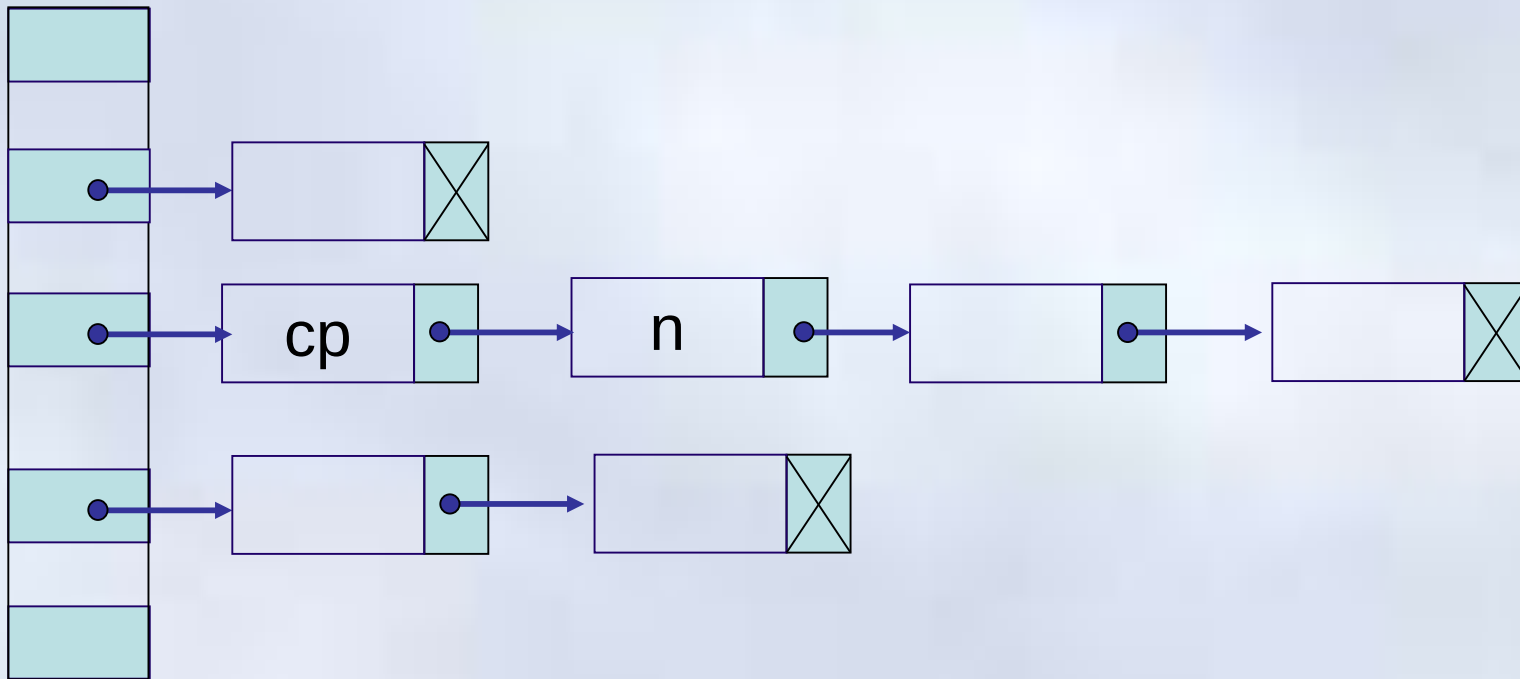
- Dùng một (*nhiều*) mảng lưu các tên trong c/trình
 - Kích thước cố định (phải lớn → lãng phí)
 - Giải quyết: danh sách liên kết (hai) chiều
 - Tiết kiệm bộ nhớ
- Danh sách không sắp xếp
 - Thêm tên vào bảng theo thứ tự gặp trong c/ trình nguồn
 - Tốc độ tìm kiếm chậm (*Độ phức tạp: $O(n)$*)
 - Thường dùng với các ngôn ngữ nhỏ, có ít danh biểu
- Danh sách sắp xếp
 - Phức tạp khi thêm tên vào bảng
 - Tốc độ tìm kiếm: *$O(\log(n))$*
 - Dùng khi tập danh biểu xác định (*VD tập từ khóa*)

Các CTDL cho bảng ký hiệu → Cây tìm kiếm

- Các nút của cây nhị phân có
 - Khóa là tên của bản ghi
 - 2 con trỏ Left và Right
- Mọi nút phải thỏa mãn
 - Khóa của con trái phải nhỏ hơn
 - Khóa của con phải phải lớn hơn
- Thời gian tìm kiếm $O(\log_2(n))$
 - Gốc rỗng → không tồn tại
 - Trùng khóa gốc → thỏa mãn
 - Nhỏ hơn → Tìm tại con trái
 - Lớn hơn → Tìm tại con phải



Các CTDL cho bảng ký hiệu→Bảng băm



Bảng là danh sách N phần tử

- Số phần tử cố định
 - Mỗi phần tử chứa tên và các thuộc tính của tên
 - Có thể là con trỏ, trỏ tới danh sách liên kết
- Đòi hỏi một hàm băm

Các CTDL cho bảng ký hiệu → Bảng băm

- H là bảng băm gồm N phần tử
 - h: là hàm băm
 - α : là một tên
- } $h(\alpha) \in [0..N-1]$

Hoạt động

Giả thiết $\beta = H[h(\alpha)]$

- $\beta = \alpha \rightarrow$ Trùng tên: Tên đã khai báo
- $\beta = \varepsilon \rightarrow$ Tên chưa khai báo
- $\beta \neq \varepsilon \ \& \ \beta \neq \alpha \rightarrow$ Xung đột. Một ô chứa 2 tên
 - Đi theo DSLK tương ứng để ra được α

Bảng ký hiệu cho cấu trúc khối

- Ngôn ngữ là có cấu trúc khối nếu
 - Một khối có thể được nằm trong khối khác
 - Ví dụ: Trong *Procedure* có *Procedure* khác...
 - Phạm vi của khai báo trong mỗi khối là chính khối đó và các khối nằm trong nó
- Luật lồng nhau gần nhất
 - Cho phép nhiều khai báo của cùng một tên
 - Các tên phải nằm trong các khối khác nhau
 - Khai báo có hiệu lực thuộc khối gần nhất
 - Khi một thủ tục nằm trong thủ tục khác được gọi, các khai báo của thủ tục bên ngoài tạm dừng hoạt động

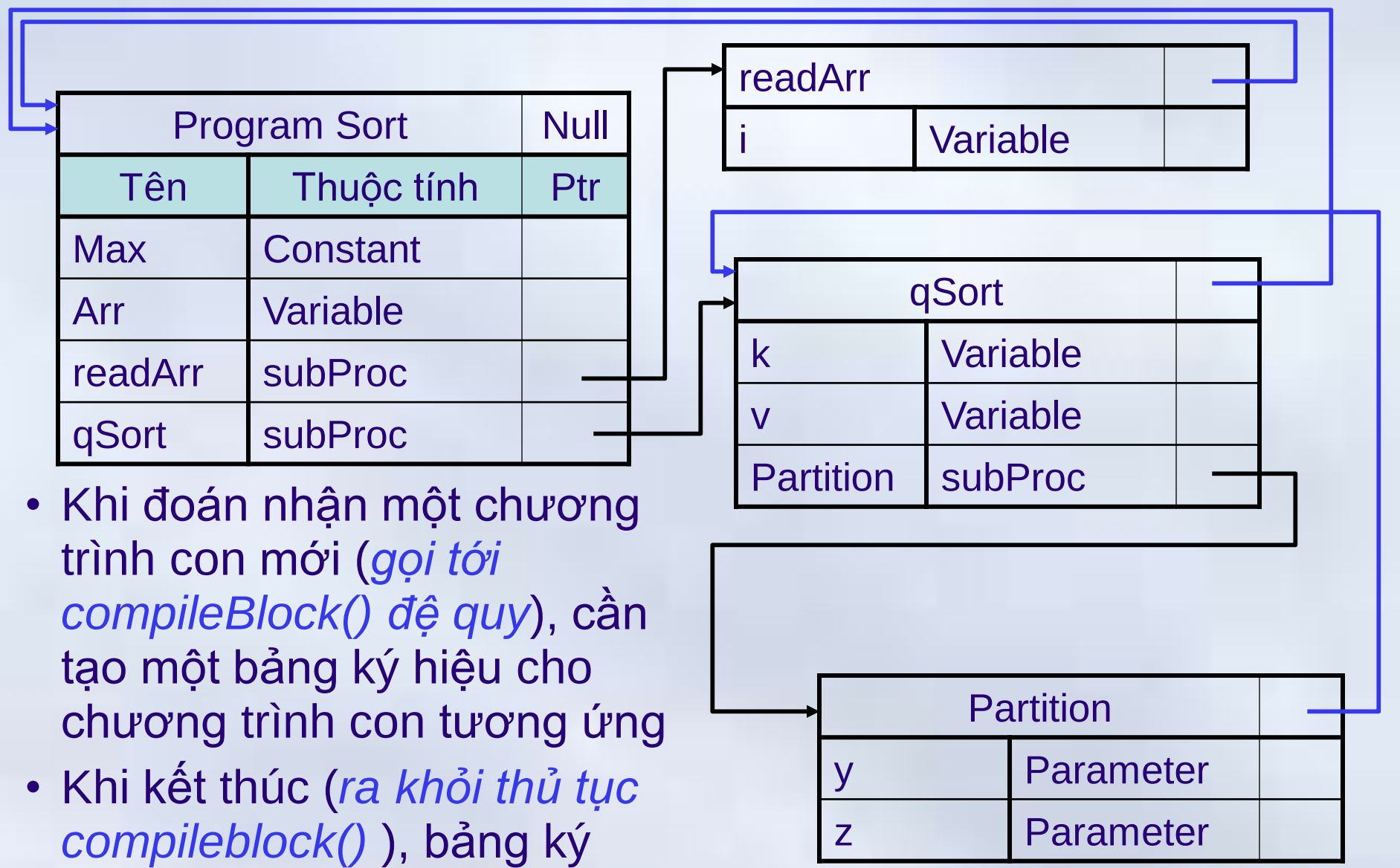
Luật về phạm vi sử dụng

- Toán tử thêm (**insert**) vào bảng ký hiệu không được ghi đè những khai báo trước
- Toán tử tìm kiếm (**lookup**) trong bảng ký hiệu luôn luôn tham chiếu luật phạm vi gần nhất
- Toán tử xóa (**delete**) chỉ được xóa khai báo gần nhất
- Giải pháp
 - Dùng stack để lưu trữ dấu vết của các thủ tục lồng nhau
 - Tạo bảng ký hiệu mới cho mỗi thủ tục
 - Thêm một định danh mới, cần chỉ rõ bảng ký hiệu cần thêm

Phạm vi → Ví dụ

- 1) **Program** sort; //Chương trình Quicksort
- 2) *Const Max = 100;*
- 3) *Var Arr: array[0..Max] of integer;*
- 4) **Procedure** readarray;
- 5) *Var i: integer;*
- 6) *Begin end {readarray};*
- 7) **Procedure** exchange(i, j: integer);
- 8) *Begin {exchange} end;*
- 9) **Procedure** quicksort(m, n: integer);
- 10) *Var k, v: integer;*
- 11) **Function** partition(y,z: integer): integer;
- 12) *Beginexchange(i,j) end; {partition}*
- 13) *Begin ... end; {quicksort}*
- 14) *Begin ... end; {sort}*

Dùng nhiều bảng ký hiệu

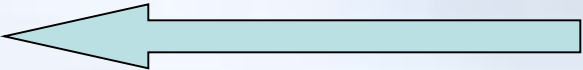


- Khi đoán nhận một chương trình con mới (*gọi tới compileBlock() đệ quy*), cần tạo một bảng ký hiệu cho chương trình con tương ứng
- Khi kết thúc (*ra khỏi thủ tục compileblock()*), bảng ký hiệu sẽ bị xóa

Dùng Stack

j	Variable
i	Variable
Partition	subProc
v	Variable
k	Variable
qSort	subProc
readArr	subProc
Max	constant
Arr	Variable
Đáy Stack	

Vị trí thêm vào



Chiều tìm kiếm



Khi gặp danh biểu
trong khai báo đưa
vào đỉnh của stack

Gặp danh biểu
trong câu lệnh, tìm
từ đỉnh trở xuống

Ví dụ đơn giản

- Dùng stack làm bảng ký hiệu
- Dùng giá trị **Tx** cho biết vị trí của đỉnh stack
 - Tx là **tham trị** của `compileBlock()`//`compileBlock(int Tx)`
 - Giá trị của Tx không thay đổi trong `compileBlock()`
 - Khi phân tích Block, thủ tục `compileBlock()` có thể gọi đến chính nó, Tx sẽ được kế thừa
 - Khi đoán nhận thủ tục con, Tx sẽ tăng dần lên khi gặp các danh biểu của chương trình con
 - Khi đoán nhận xong - thoát ra khỏi `compileBlock()`, Tx khôi phục lại giá trị cũ (*giá trị trước khi gọi*)
 - Nhưng khai báo bên trong một khối sẽ bị xóa đi
 - Ban đầu, danh sách rỗng → `compileBlock(0)`

Ví dụ đơn giản

```
Const M=10;  
Var X, Y: integer;  
Procedure A;  
  Var X, Z: Integer;  
  Procedure AA;  
    Var X: integer;  
    Begin  
      Z= X+Y*M  
    End;  
  Begin  
  End;  
Procedure B;  
  Var X, Y;  
  Begin  
  End;  
Begin  
End
```

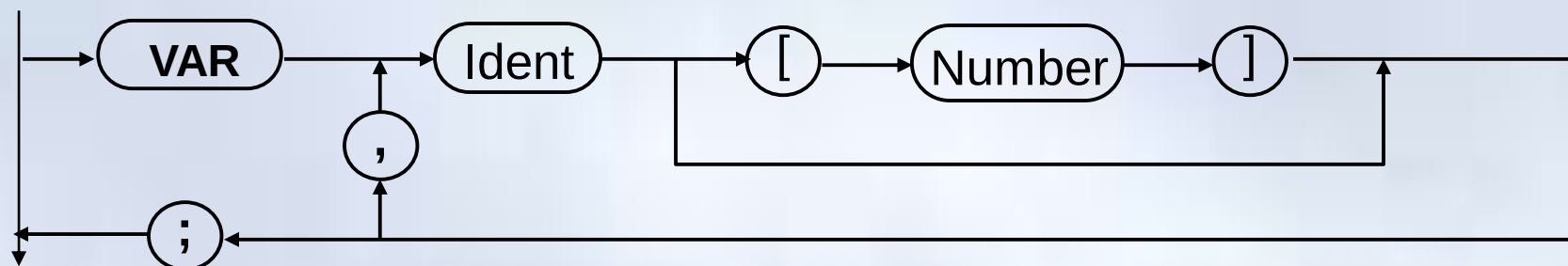
X	Variable
AA	subProc
Z	Variable
X	Variable
A	subProc
Y	constant
X	constant
M	Constant
Đáy Stack	

Ví dụ đơn giản

Cần các thủ tục

- **Procedure Enter(char * Id, Object Type);**
 - Đưa một danh biểu vào bảng ký hiệu
 - Giá trị của danh biểu
 - Kiểu danh biểu (constant, type, variable, procedure, function)
- **Function Location(char * Id) : int;**
 - Chỉ ra vị trí của danh biểu trong bảng ký hiệu. Nếu không thấy, trả về giá trị 0
- **Function checkIdent(char * Id) : Boolean;**
 - Kiểm tra tên (Id) đã được khai báo trong phạm vi chưa
- **Function getKind(int idx) : Object**
 - Trả về kiểu của danh biểu tại vị trí idx

Ví dụ đơn giản → Khai báo



```
void compileBlock( ){
```

```
.....
```

```
if(Token==VAR){
```

```
    Token = getToken();
```

```
    compileDeclareVariable ();
```

```
    while(Token == COMMA) {
```

```
        Token = getToken();
```

```
        compileDeclareVariable ();
```

```
    };
```

```
}
```

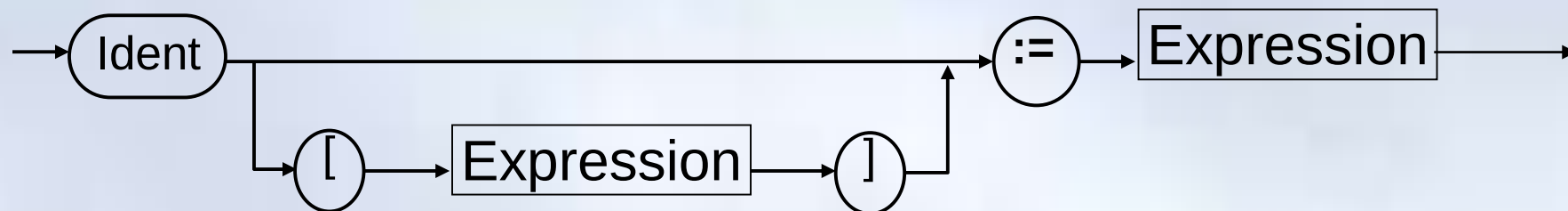
```
.....
```

```
}
```

Ví dụ đơn giản→Khai báo

```
void compileDeclareVariable(void){  
    if(Token==IDENT){  
        if(CheckIdent(Id) == 0) Enter(Id,Variable);  
        Else  Error();//Trung ten  
        Token = getToken();  
        if(Token == LBRACK){  
            .....  
        }  
    }else Error(19);//Thieu ten bien  
}  
}
```

Ví dụ đơn giản → Sử dụng



```

void compileStatement(){
    int p;
    if(Token == IDENT){
        p = Location(Id);
        if(p < 0) Error(52); //Ten chua khai bao
        if(getKind(p) != Variable)
            Error( );//Ve trai phai la mot bien
        Token = getToken();
        if(Token == LBRACK){
            .....
        }
    }
}
  
```

Chương 4: Phân tích ngữ nghĩa

1. Giới thiệu
2. Bảng ký hiệu
3. Chương trình dịch định hướng cú pháp
4. Kiểm tra kiểu
5. Xử lý sai sót

Định nghĩa hướng cú pháp (Syntax directed definition)

- Là dạng tổng quát của VPPNC
- Mỗi ký hiệu VP liên kết với một tập thuộc tính
 - Hai loại thuộc tính: Tổng hợp và kế thừa
 - Thuộc tính là khái niệm trừu tượng, biểu diễn một đại lượng bất kỳ: số, xâu, vị trí trong bộ nhớ..
- **Thuộc tính tổng hợp**
 - Giá trị của thuộc tính tại một nút trong cây được xác định từ giá trị của các nút con của nó.
- **Thuộc tính kế thừa**
 - Giá trị của thuộc tính được định nghĩa dựa vào giá trị nút cha và/hoặc các nút anh em của nó.

Dạng của định nghĩa hướng cú pháp

- Mỗi sản xuất dạng $A \rightarrow \alpha$ liên hệ với một tập luật ngữ nghĩa có dạng $b = f(c_1, c_2, \dots, c_n)$ trong đó f là một hàm và b thoả một trong hai yêu cầu sau:
 - b là **thuộc tính tổng hợp** của A ; $c_1, \dots, c_n$ là các thuộc tính của các ký hiệu trong vế phải sản xuất $A \rightarrow \alpha$
 - b là **thuộc tính kế thừa** của một trong những ký hiệu trong α ; $c_1, \dots, c_n$ là thuộc tính của các ký hiệu trong sản xuất $A \rightarrow \alpha$
- Tập luật ngữ nghĩa dùng để tính giá trị thuộc tính của ký hiệu A

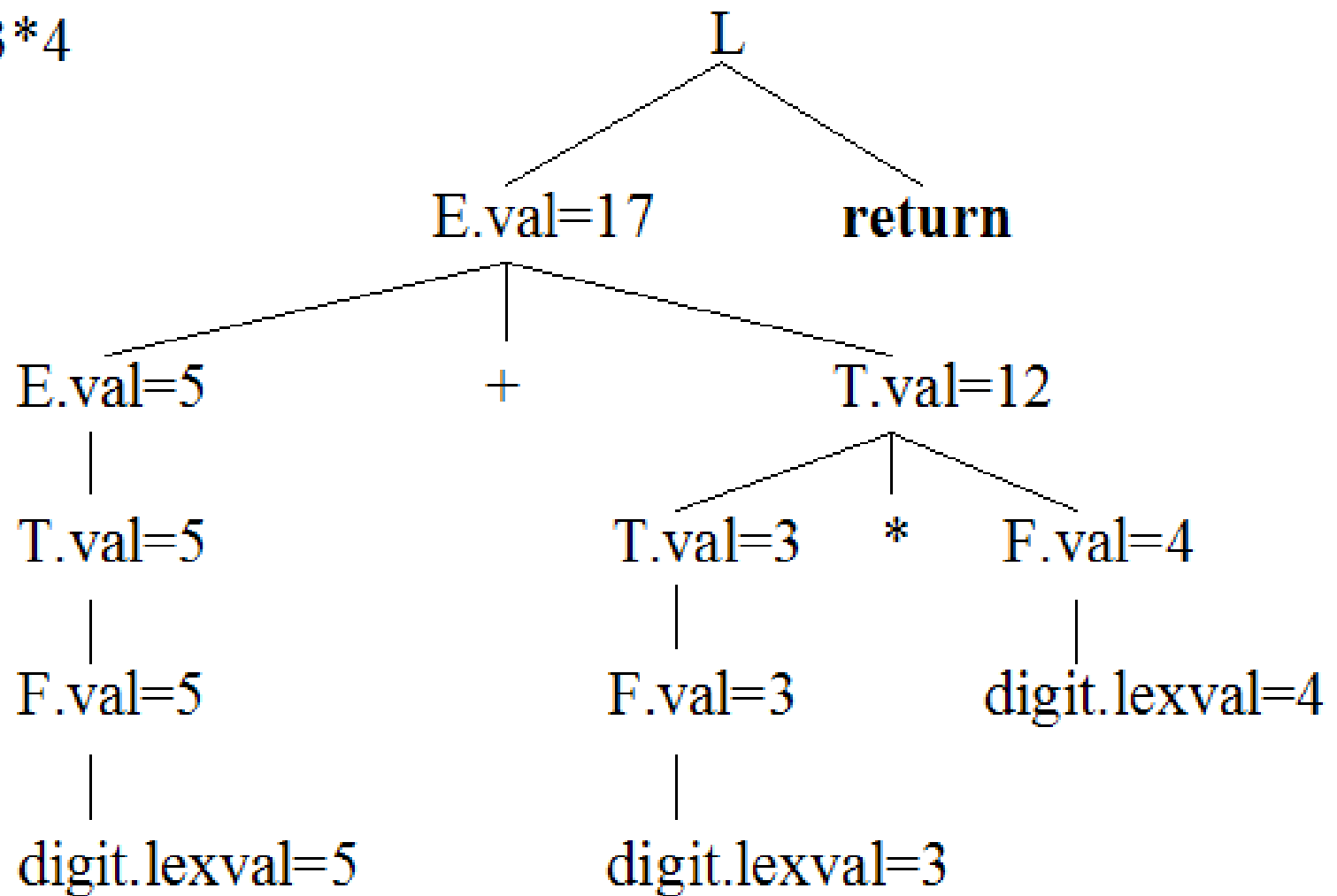
Định nghĩa hướng cú pháp → Ví dụ

Sản xuất	Quy tắc ngữ nghĩa
$L \rightarrow E \text{ return}$	Print ($E.val$)
$E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$
$E \rightarrow T$	$E.val = T.val$
$T \rightarrow T_1 * F$	$T.val = T_1.val * F.val$
$T \rightarrow F$	$T.val = F.val$
$F \rightarrow (E)$	$F.val = E.val$
$F \rightarrow \text{digit}$	$F.val = \text{digit.lexval}$
• Các ký hiệu <i>E, T, F</i> có thuộc tính tổng hợp <i>val</i> • Từ tố <i>digit</i> có thuộc tính tổng hợp <i>lexval</i> (Được bộ phân tích từ vựng đưa ra)	

Cây phân tích cú pháp có chú giải

Cây có chỉ ra giá trị các thuộc tính tại mỗi nút

Input: 5+3*4



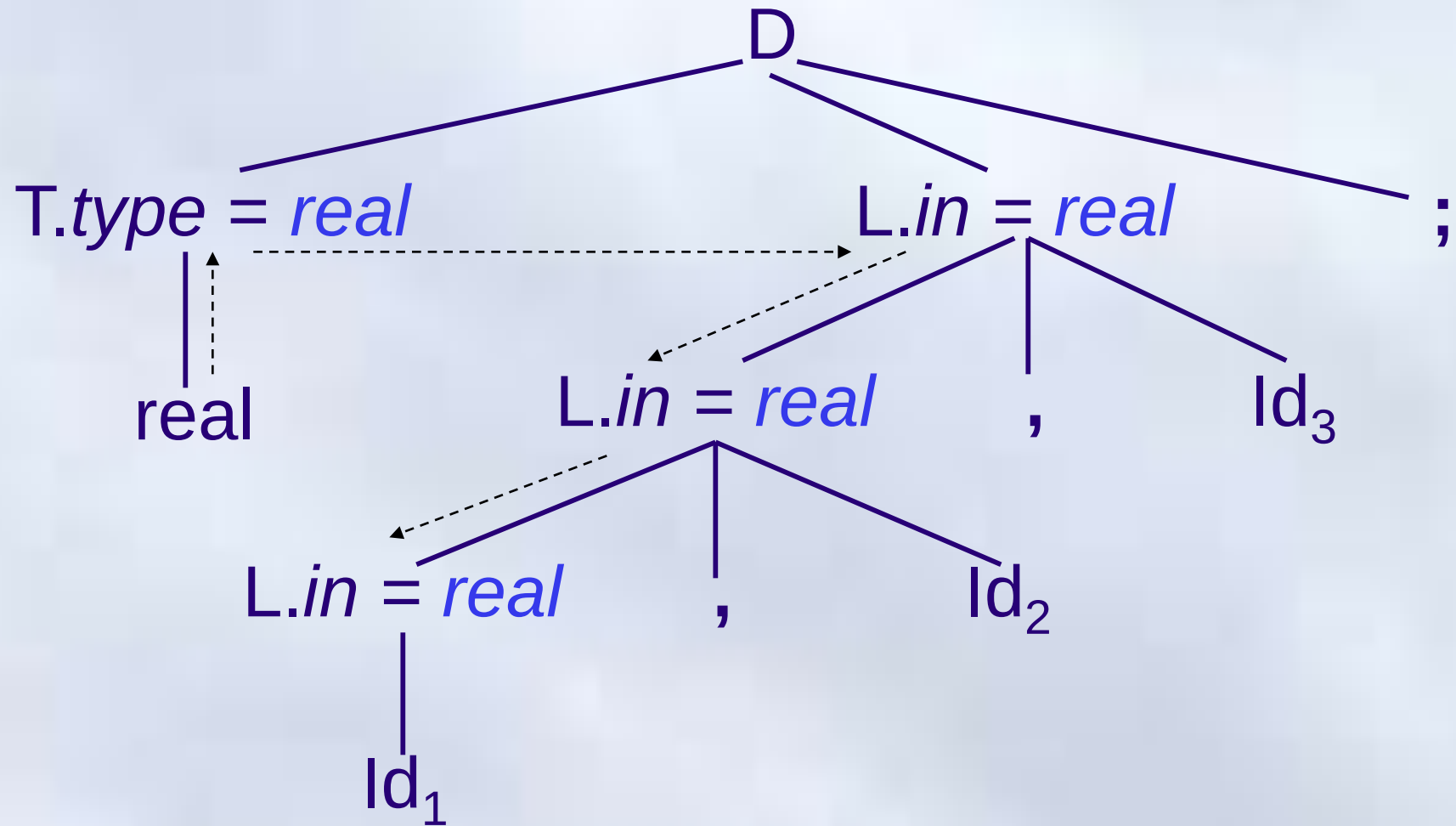
Thuộc tính kế thừa → Ví dụ

Sản xuất	Quy tắc ngữ nghĩa
$D \rightarrow T L ;$	$L.in := T.type$
$T \rightarrow int$	$T.type = integer$
$T \rightarrow float$	$T.type := real$
$L \rightarrow L_1 , Id \quad // L_1 \equiv L$	$L_1.in := L.in$ $addType(Id.entry, L.in)$
$L \rightarrow Id$	$addType(Id.entry, L.in)$

- Ký hiệu không kết thúc **T** có thuộc tính tổng hợp *type* có giá trị được xác định bởi từ khóa *int/float* trong khai báo
- Luật ngữ nghĩa $L.in := T.type$ gắn với sản xuất $D \rightarrow T L ;$ ấn định thuộc tính kế thừa của **L** (*L.in*) tới kiểu khai báo
- Luật ngữ nghĩa gắn với sản xuất từ **L**, sẽ truyền kiểu của **L** với các khai báo tiếp và dùng thủ tục *addType()* để lưu thuộc tính kiểu của danh biểu vào bảng ký hiệu tại vị trí *Id.entry*

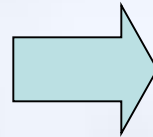
Cây phân tích cú pháp có chú giải

Ví dụ với khai báo: float x, y, z ;

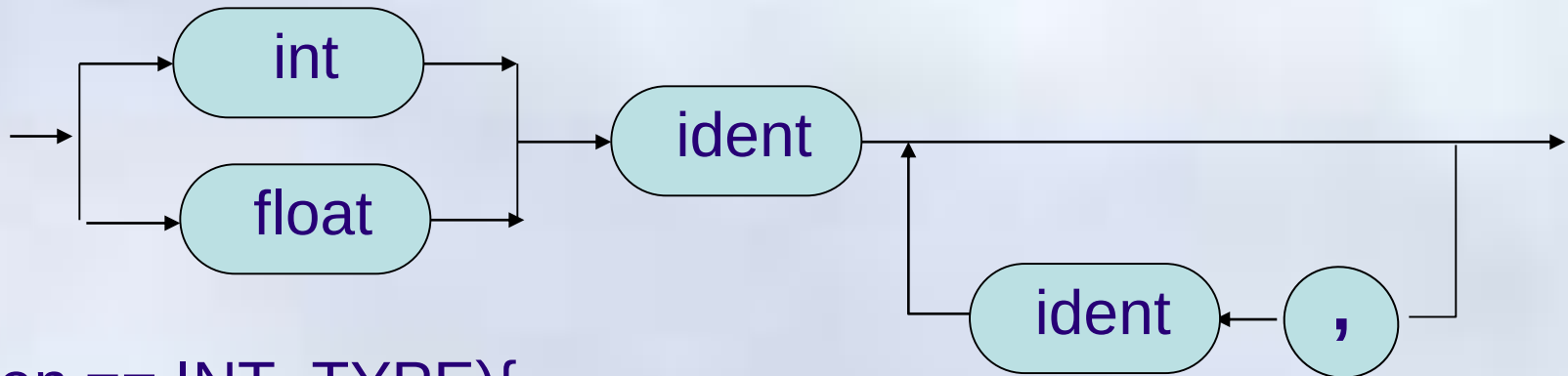


Thuộc tính kế thừa → Ví dụ

- $D \rightarrow T L$
- $T \rightarrow \text{int} \mid \text{float}$
- $L \rightarrow L , \text{Ident} \mid \text{Ident}$



- $D \rightarrow T L$
- $T \rightarrow \text{int} \mid \text{float}$
- $L \rightarrow \text{Ident} \{ , \text{Ident} \}$



```

if(Token == INT_TYPE){
    Type = INT_TYPE;
    Token= getToken();
    if (Token = IDENT)
  
```

```

    if(CheckIdent(Id) == 0) Enter(Id,Variable,INT_TYPE);
  
```

Lưu trữ thông tin về phạm vi

- Văn phạm cho phép các chương trình con bao nhau
 - Khi bắt đầu phân tích chương trình con, phần khai báo của chương trình bao tạm dừng
 - Dùng một bảng ký hiệu riêng cho mỗi chương trình con
- Văn phạm của khai báo này:

$P \rightarrow D ; S$ //D: Description, S : Statement

$D \rightarrow D ; D \mid id : T \mid \text{proc } id ; D_1 ; S$ // $D_1 \equiv D$

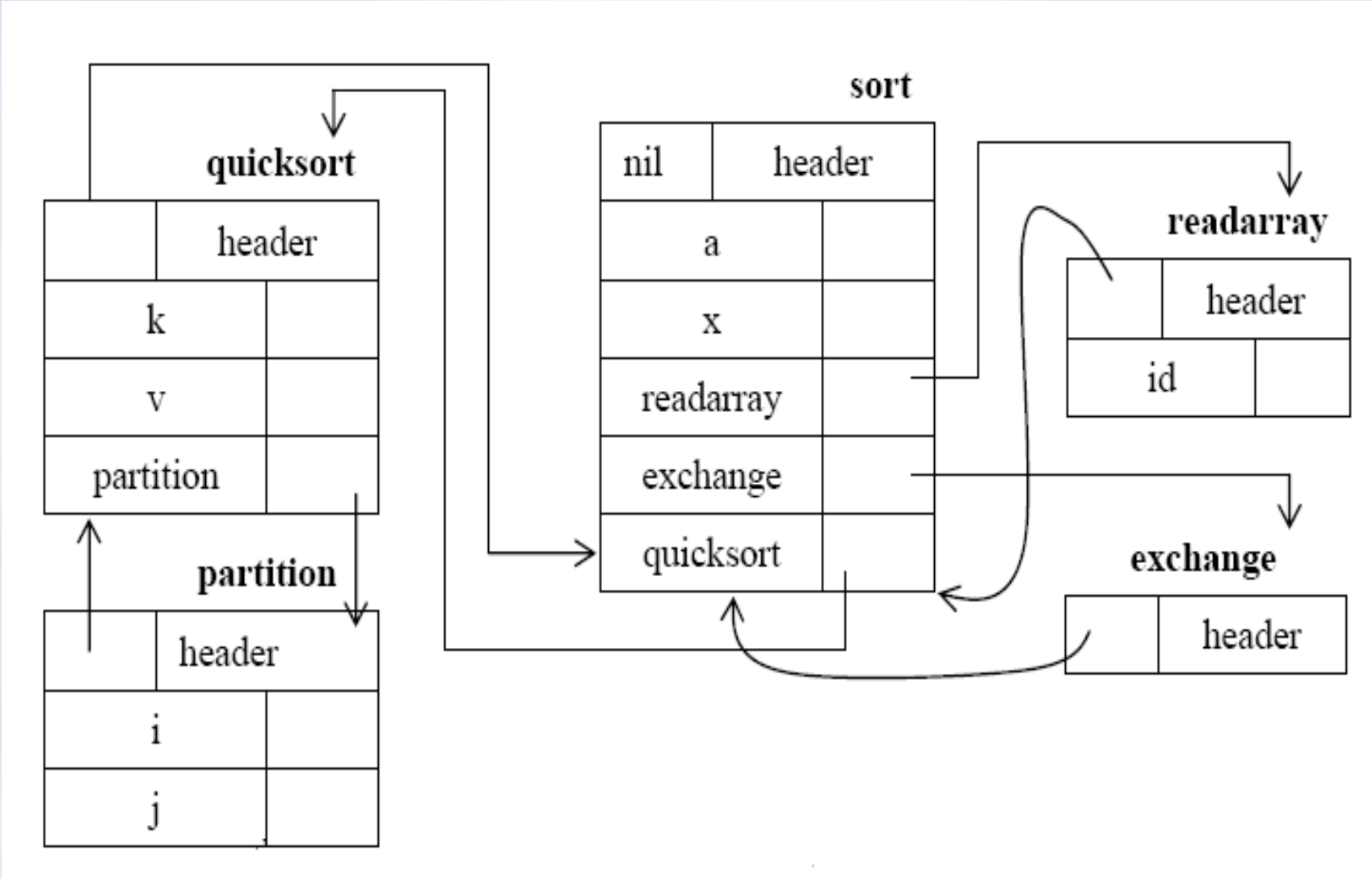
- Khi khai báo chương trình con $D \rightarrow \text{proc } id D_1 ; S$ được phân tích thì các khai báo trong D_1 được lưu trong bảng ký hiệu mới.

Lưu trữ thông tin về phạm vi → Ví dụ

```
1) Program sort;    //Chương trình Quicksort
2) Var a: array[0..10] of integer;
3)    x: integer;
4)    Procedure readarray;
5)        Var i: integer;
6)        Begin ..... end {readarray};
7)    Procedure exchange(i, j: integer);
8)        Begin {exchange} end;
9)    Procedure quicksort(m, n: integer);
10)        Var k, v: integer;
11)        Function partition(y,z: integer): integer;
12)            Var l, j : integer;
13)            Begin .....exchange(i,j) end; {partition}
14)    Begin ... end; {quicksort}
15) Begin ... end; {sort}
```

Lưu trữ thông tin về phạm vi→Ví dụ

- Các bảng ký hiệu của chương trình sort



Quy tắc ngữ nghĩa → Các thao tác

- **mktable(previous)** – tạo một bảng kí hiệu mới, bảng này có *previous* chỉ đến bảng cha của nó.
- **enter(table,name,type,offset)** – thêm một đối tượng mới có tên *name* vào bảng kí hiệu được chỉ ra bởi *table* và đặt kiểu là *type* và địa chỉ tương đối là *offset* vào các trường tương ứng.
- **enterproc(table,name,newbtable)** – tạo một phần tử mới trong bảng *table* cho chương trình con *name*, *newbtable* trỏ tới bảng kí hiệu của CTC này.
- **addwidth(table,width)** – ghi tổng kích thước của tất cả các p/tử trong bảng kí hiệu vào header của bảng.

Khai báo trong chương trình con

Sản xuất	Quy tắc ngữ nghĩa
$P \rightarrow MD ; S$	<i>addwidth(top(tblptr), top(offset)); pop(tblptr); pop(offset)</i>
$M \rightarrow \varepsilon$	<i>t:=mktable(null); push(t, tblptr); push(0, offset)</i>
$D \rightarrow D ; D$	
$D \rightarrow \text{proc id}; ND_1; S$	<i>t:=top(tblptr); addwidth(t, top(offset)); pop(tblptr); pop(offset); enterproc(top(tblptr), id.name, t)</i>
$N \rightarrow \varepsilon$	<i>t:=mktable(top(tblptr)); push(t, tblptr); push(0, offset);</i>
$D \rightarrow id : T$	<i>enter(top(tblptr), id.name, T.type, top(offset); top(offset):=top(offset) + T.width</i>

Tblptr: là Stack dùng chứa các con trỏ trỏ tới bảng ký hiệu

Offset: Là Stack dùng lưu trữ các Offset

Xử lý các khai báo trong chương trình con

- Sản xuất: $P \rightarrow MD$:
 - Hoạt động của cây con M được thực hiện trước
- Sản xuất: $M \rightarrow \varepsilon$:
 - Tạo bảng ký hiệu cho phạm vi ngoài cùng (*chương trình sort*) bằng lệnh ***mktable(nil)*** // *Không có SymTab cha*
 - Khởi tạo stack ***tblptr*** với bảng ký hiệu vừa tạo ra
 - Đặt offset = 0.
- Sản xuất: $N \rightarrow \varepsilon$:
 - Tạo ra một bảng mới ***mktable(top(tblptr))***
 - Tham số ***top(tblptr)*** cho giá trị con trỏ tới bảng cha
 - Thêm bảng mới vào đỉnh stack ***tblptr*** // *push(t,tblptr)*
 - 0 được đẩy vào stack ***offset*** // *push(0,Offset)*

N đóng vai trò tương tự M khi một khai báo CTC xuất hiện

Xử lý các khai báo trong chương trình con

- Với mỗi khai báo **id: T**
 - một phần tử mới được tạo ra cho **id** trong bảng kí hiệu hiện hành (*top(tblptr)*)
 - Stack *tblptr* không đổi,
 - Giá trị **top(offset)** được tăng lên bởi **T.width**.
- Khi **D → proc id ; N D₁ ; S** diễn ra
 - Kích thước của tất cả các đối tượng dữ liệu khai báo trong **D₁** sẽ nằm trên đỉnh stack **offset**.
 - Kích thước này được lưu trữ bằng cách dùng **Addwidth()**,
 - Các stack *tblptr* và **offset** bị lấy mất phần tử trên cùng (**pop()**)
 - Thao tác thực hiện trên các khai báo của chương trình con.

Ví dụ

```
X: float;  
Y: Int;  
Proc A;  
    X:int;  
    N : float;  
    Proc B;  
        X:int;  
        SB  
    SA  
S
```

$P \rightarrow D ; S$

$D \rightarrow D; D \mid id : T \mid \text{proc } id ; D_1 ; S$

- Xây dựng cây phân tích có chú giải cho chương trình hình bên.
- Xây dựng bảng ký hiệu, có chỉ rõ vị trí các biến

Chương 4: Phân tích ngữ nghĩa

1. Giới thiệu
2. Bảng ký hiệu
3. Chương trình dịch định hướng cú pháp
4. Kiểm tra kiểu
5. Xử lý sai sót

Kiểm tra kiểu

- Có hai phương pháp kiểm tra kiểu
- Kiểm tra tĩnh : áp dụng trong thời gian dịch
 - Trong nhiều ngôn ngữ (C hay Pascal,..) kiểm tra kiểu là tĩnh và được dùng để kiểm tra tính đúng đắn của chương trình trước khi nó được thực hiện
- Kiểm tra động : thực hiện trong thời gian thực thi
 - Một số phép kiểm tra chỉ có thể thực hiện động
 - `int Tab[255], i;` → khi thực hiện `Tab[i]` có thể nằm ngoài phạm vi
- Bộ kiểm tra kiểu được xây dựng dựa trên
 - Các **biểu thức kiểu** của ngôn ngữ
 - Bộ **luật để định kiểu** cho các cấu trúc

Biểu thức kiểu (type expression)

Là một kiểu dữ liệu chuẩn hoặc được xây dựng từ các kiểu dữ liệu khác bởi cấu trúc kiểu

1. Kiểu cơ sở (*boolean, char, integer, real*) là biểu thức kiểu.

Chấp nhận các kiểu cơ sở đặc biệt:

- **type_error**: chỉ ra một lỗi trong quá trình kiểm tra kiểu
- **void**: cho phép kiểm tra kiểu đối với lệnh.

2. Do biểu thức kiểu có thể được đặt tên $\Rightarrow$ Tên là biểu thức kiểu

3. Áp dụng toán tử xây dựng kiểu (*cấu trúc kiểu*) vào các biểu thức kiểu tạo ra biểu thức kiểu

Cấu trúc kiểu

- Mảng (Array):

- Nếu **T** là biểu thức kiểu thì **array(I,T)** là biểu thức kiểu biểu diễn một mảng với các phần tử kiểu **T** và chỉ số trong miền **I**

- Khai báo: array [10] of integer
- có kiểu: array(1..10,int); //array(10,int)
- Ví dụ: Array[10] of Array[20] of integer
 - Array(10,Array(20,int))

- Tích Descarter:

- Nếu **T₁** và **T₂** là các biểu thức kiểu thì tích Descarter **T₁ × T₂** là biểu thức kiểu

Cấu trúc kiểu

- **Bản ghi (*Record*):**

- Là tích Descartes các kiểu của các trường bên trong
- Ví dụ:

```
struct{  
    double r;  
    int i;  
}
```

Có kiểu $((r \times \text{double}) \times (i \times \text{int}))$

Cấu trúc kiểu

- **Con trỏ:**

- Nếu T là biểu thức kiểu thì $\text{pointer}(T)$ là biểu thức kiểu

- **Hàm**

- Nếu D là miền xác định và R là miền giá trị của hàm thì kiểu của hàm được biểu diễn là $D \rightarrow R$.

Ví dụ

- $\text{int } f(\text{char } a, \text{char } b) \rightarrow$ Có kiểu: $\text{char} \times \text{char} \rightarrow \text{int}$.
- $\text{int}^* f(\text{int } a, \text{int } b) \rightarrow$ có kiểu $\text{int} \times \text{int} \rightarrow \text{pointer}(\text{int})$

Hệ thống kiểu

- Tập các luật để ấn định các biểu thức kiểu vào các phần khác nhau của chương trình
- Bộ kiểm tra kiểu thực hiện một hệ thống kiểu
- Được định nghĩa thông qua định nghĩa tựa cú pháp

Bộ kiểm tra kiểu đơn giản

Xét văn phạm

$$P \rightarrow D;E$$

$$D \rightarrow D;D \mid \text{id} : T$$

$$T \rightarrow \text{char} \mid \text{int} \mid \text{array}[\text{num}] \text{ of } T \mid ^T$$

$$E \rightarrow \text{literal} \mid \text{num} \mid \text{id} \mid E \text{ mod } E \mid E[E] \mid E^{\wedge}$$

Văn phạm sinh ra chương trình bắt đầu bởi các khai báo D và sau đó là một biểu thức E

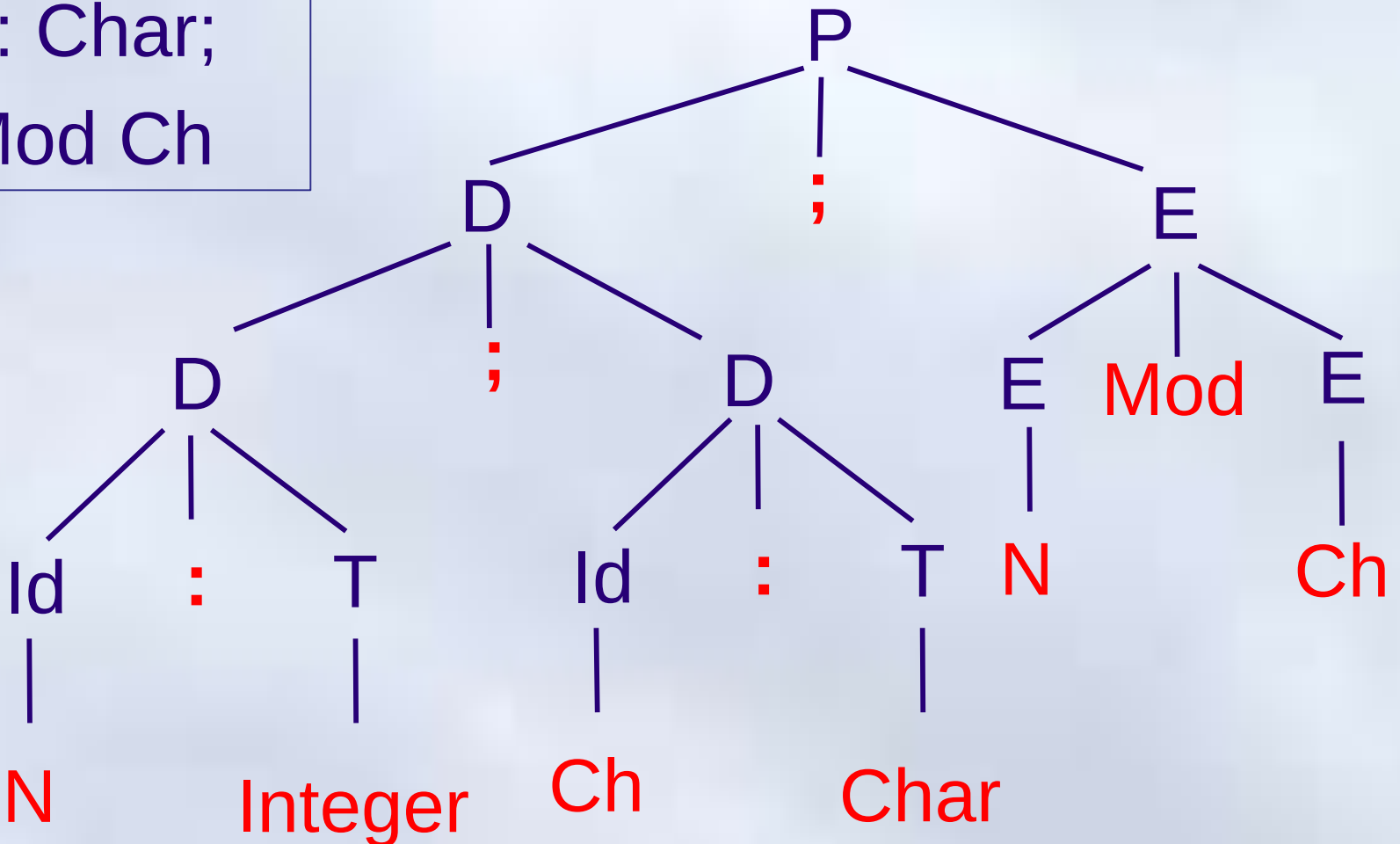
- Giả thiết chỉ số mảng bắt đầu từ 1

Bộ kiểm tra kiểu của định danh → Ví dụ

```
N : integer;
```

Ch : Char;

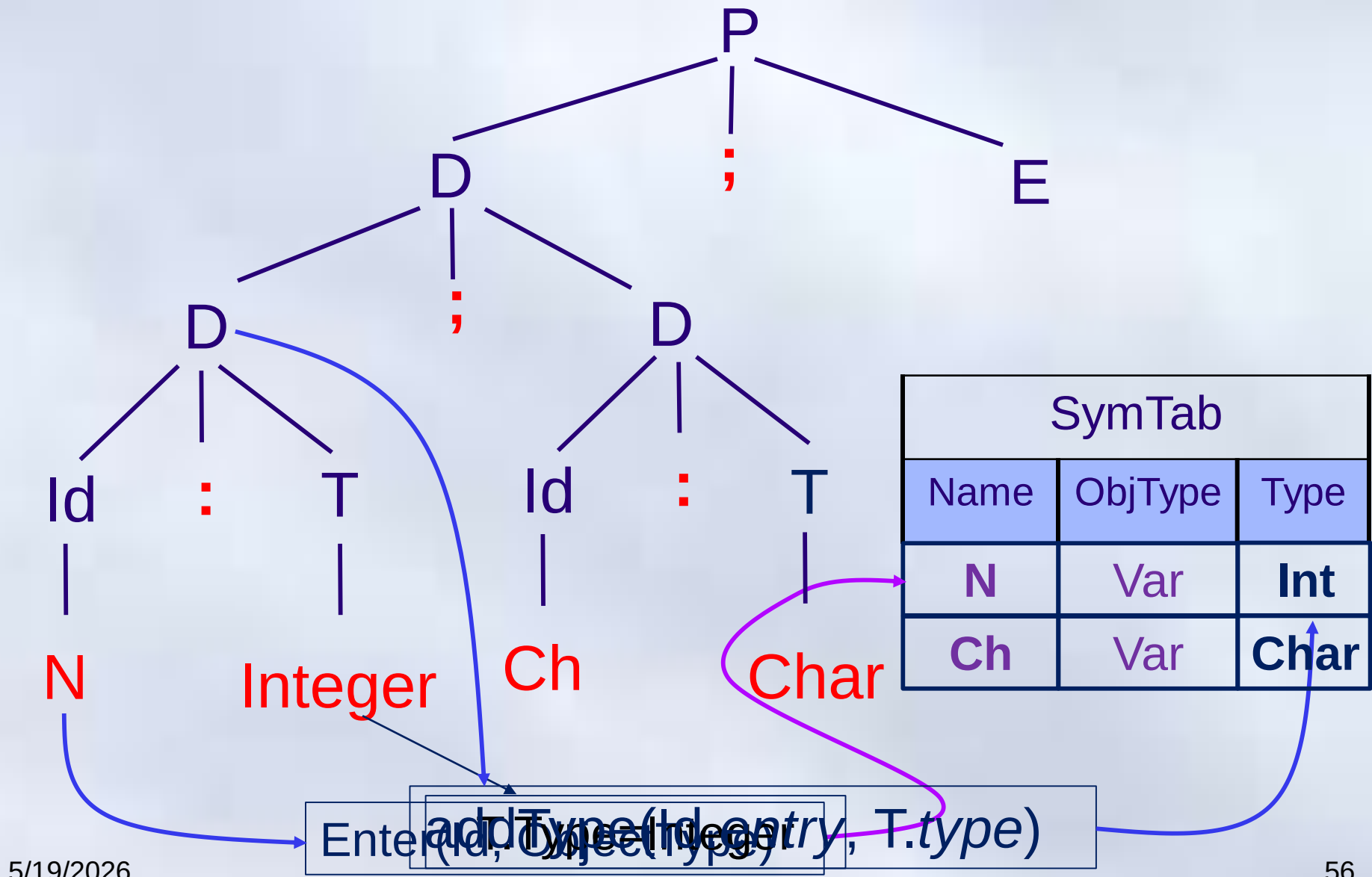
N Mod Ch



Bộ kiểm tra kiểu của định danh

Sản xuất	Quy tắc ngữ nghĩa
$P \rightarrow D ; E$	
$D \rightarrow D ; D$	
$D \rightarrow Id : T$	$addType(Id.entry, T.type)$
$T \rightarrow char$	$T.type = char$
$T \rightarrow integer$	$T.type = integer$
$T \rightarrow ^T_1$	$T.type = pointer(T_1.type)$
$T \rightarrow array[num] \text{ of } T_1$	$T.Type = array(1..num.val, T_1.type)$
Do $P \rightarrow D ; E$ nên kiểu của các định danh được khai báo đều được lưu trong bảng ký hiệu trước khi biểu thức E được kiểm tra kiểu	

Bộ kiểm tra kiểu của định danh→Ví dụ



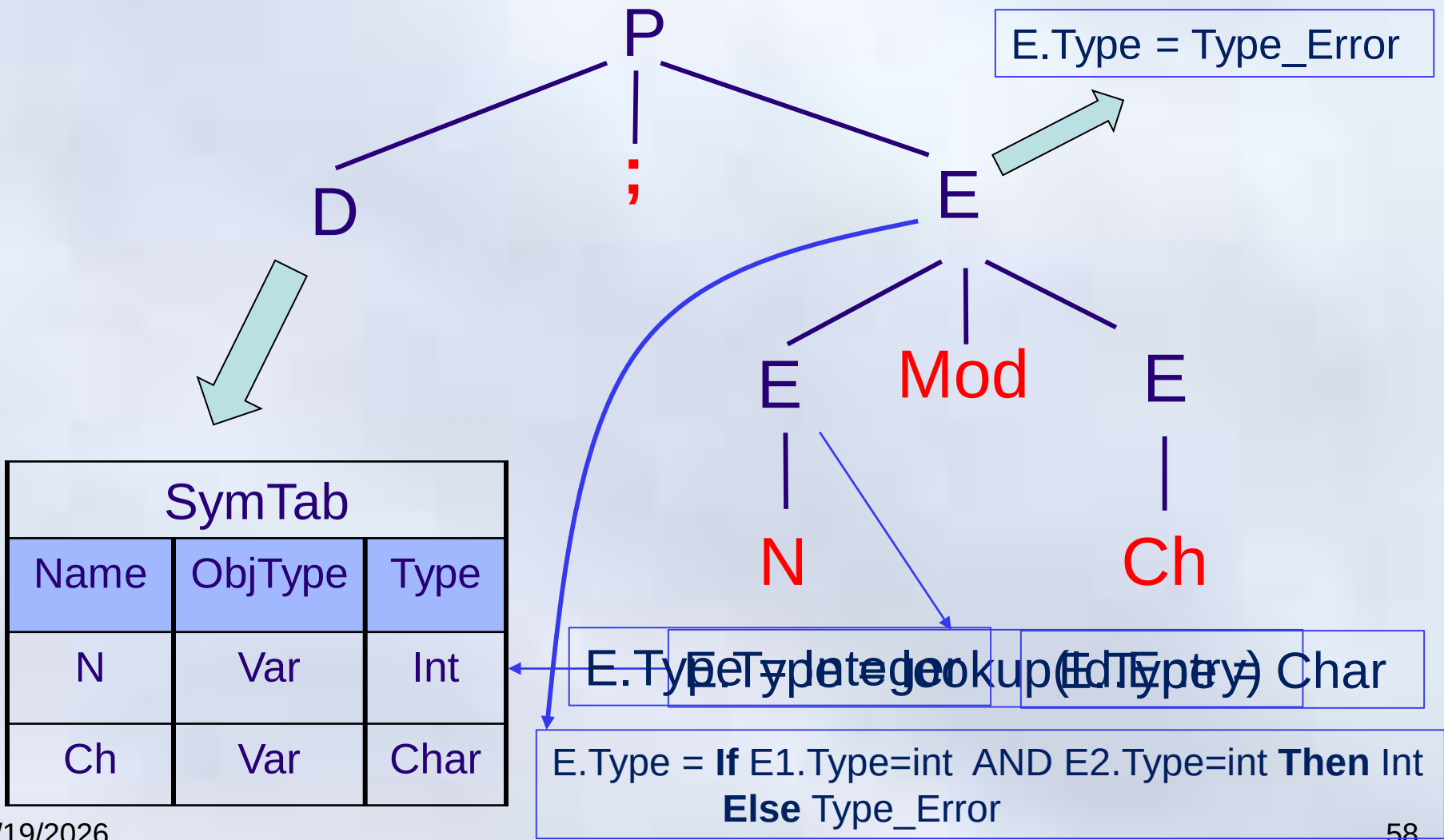
Bộ kiểm tra kiểu của biểu thức

SẢN XUẤT	QUY TẮC NGŨ NGHĨA
$E \rightarrow \text{literal}$	$E.type := \text{char}$
$E \rightarrow \text{num}$	$E.type := \text{int}$
$E \rightarrow \text{id}$	$E.type := \text{lookup}(\text{id.entry})$
$E \rightarrow E_1 \text{ mod } E_2$	$E.type := \text{if } E_1.type = \text{int} \text{ and } E_2.type = \text{int}$ then int else type_error
$E \rightarrow E_1[E_2]$	$E.type := \text{if } E_2.type = \text{int} \text{ and } E_1.type = \text{array}(s,t)$ then t else type_error
$E \rightarrow E_1 \uparrow$	$E.type := \text{if } E_1.type = \text{pointer}(t) \text{ then } t$ else type_error

Hàm `lookup(idx)` trả về kiểu trong bảng ký hiệu tại vị trí `idx`

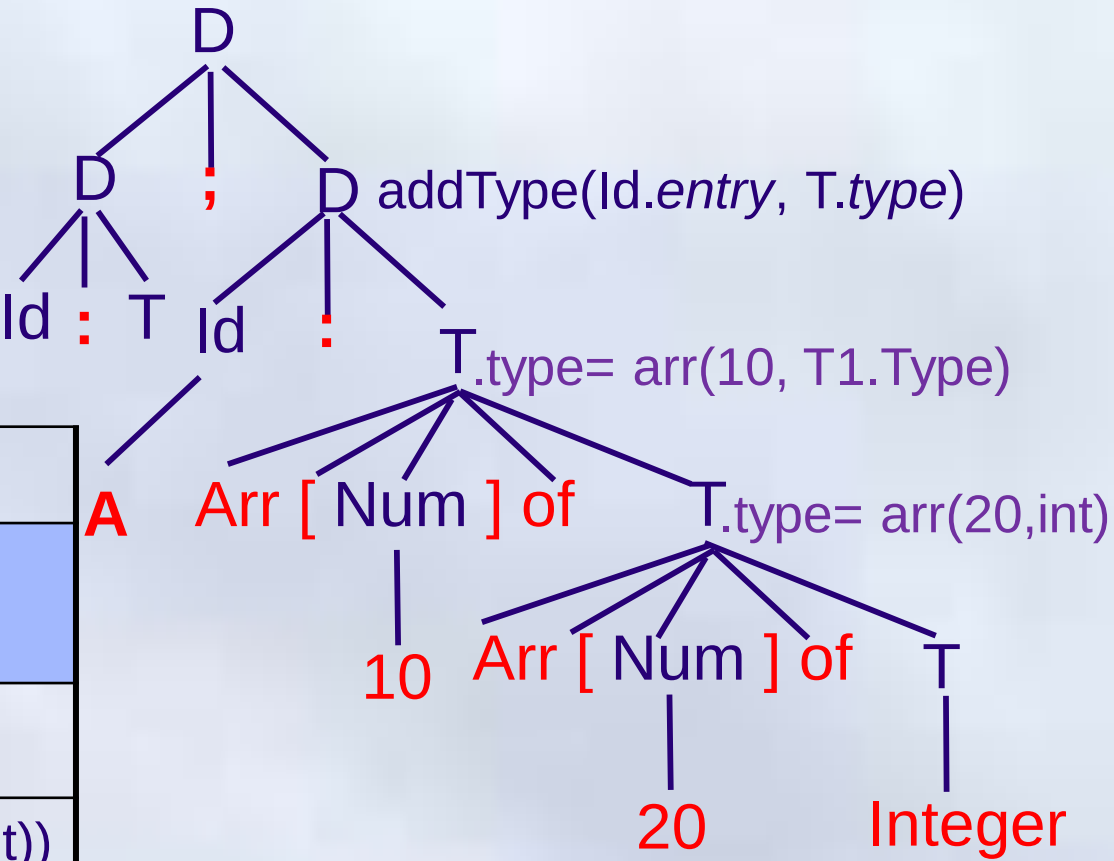
Bộ kiểm tra kiểu của định danh→Ví dụ

N : integer; Ch : Char; N Mod Ch



Bộ kiểm tra kiểu của định danh→Ví dụ 2

```
N : integer;  
A : Array[10] of Array[20] of Integer;  
A[10][10] Mod 100
```

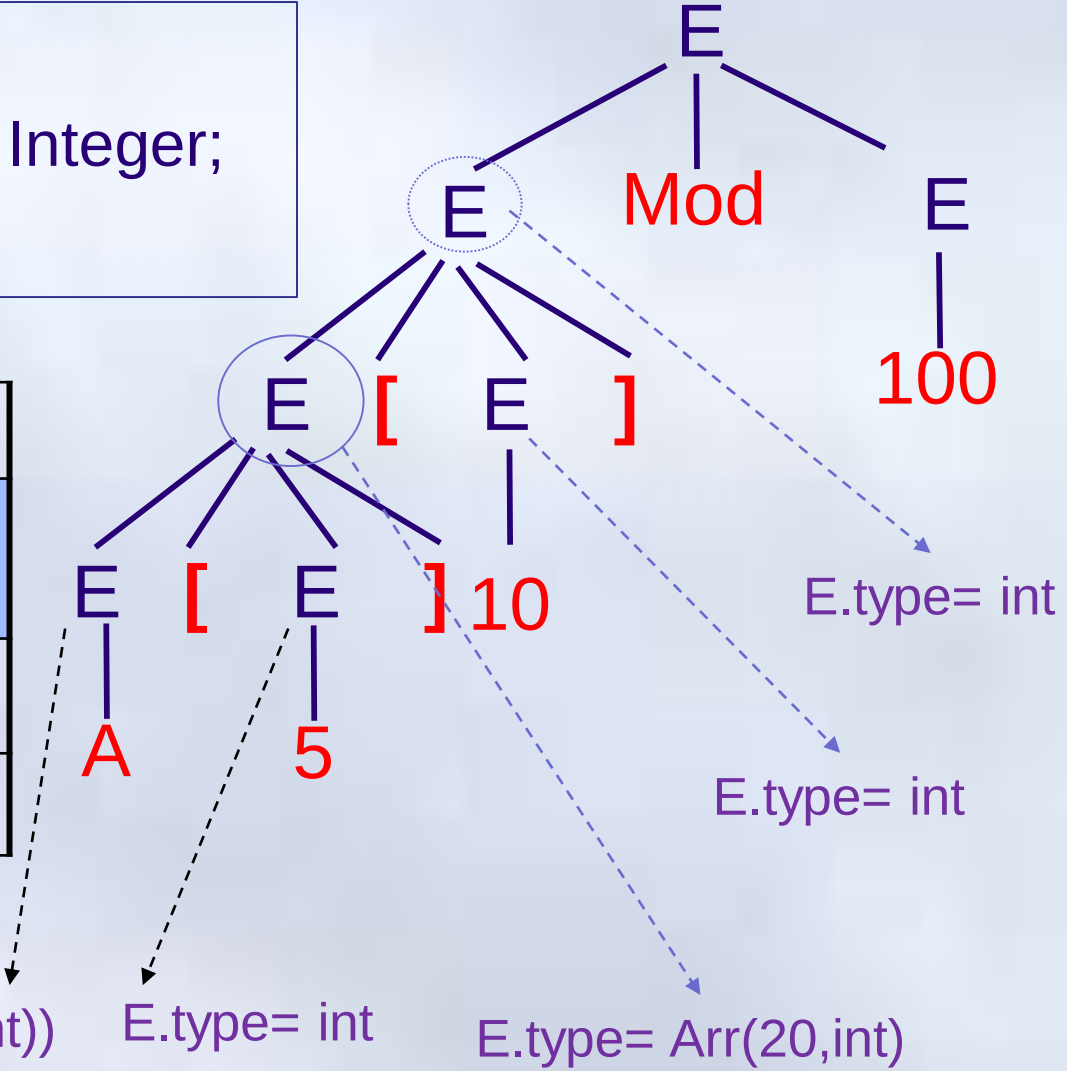


SymTab		
Nam e	Obj Type	Type
N	Var	Int
A	Var	Arr (10,Arr(20,int))

Bộ kiểm tra kiểu của định danh→Ví dụ 2

```
N : integer;  
A : Array[10] of Array[20] of Integer;  
A[5][10] Mod 100
```

SymTab		
Nam e	Obj Type	Type
N	Var	Int
A	Var	Arr (10,Arr(20,int))



Bộ kiểm tra kiểu của câu lệnh

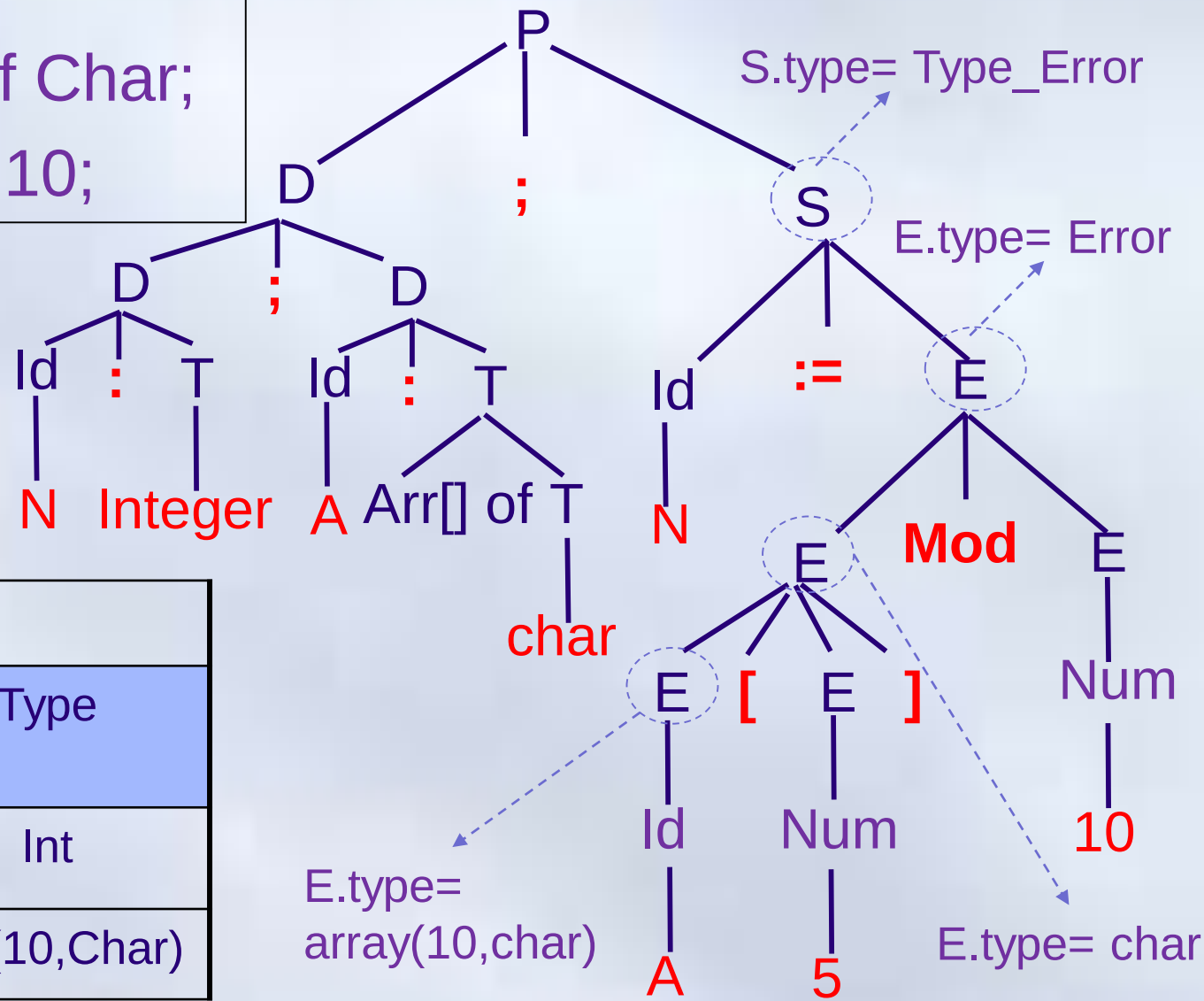
- Với các câu lệnh không có giá trị (*lệnh gán, lệnh gộp,..*) sẽ có kiểu cơ sở **void**
- Lỗi trong câu lệnh thì kiểu của lệnh là: **type_error**
- Một chương trình hoàn chỉnh có thêm luật **P → D; S**

SẢN XUẤT	QUY TẮC NGŨ NGHĨA
$S \rightarrow id := E$	$S.type := \text{if } id.type = E.type \text{ then } void$ $\text{else } type_error$
$S \rightarrow \text{if } E \text{ then } S_1$	$S.type := \text{if } E.type = boolean \text{ then } S_1.type$ $\text{else } type_error$
$S \rightarrow \text{while } E \text{ do } S_1$	$S.type := \text{if } E.type = boolean \text{ then } S_1.type$ $\text{else } type_error$
$S \rightarrow S_1; S_2$	$S.type := \text{if } S_1.type = void \text{ and } S_2.type = void$ $\text{then } void$ $\text{else } type_error$

Bộ kiểm tra kiểu của câu lệnh → Ví dụ

```
N : integer;  
A : Array[10] of Char;  
N := A[5] Mod 10;
```

SymTab		
Na me	Obj Type	Type
N	Var	Int
A	Var	Arr (10,Char)



Bộ kiểm tra kiểu của hàm

Sản xuất	Quy tắc ngữ nghĩa
$D \rightarrow Id : T$	$\text{addType}(Id.entry, T.type)$ $D.type = T.type$
$D \rightarrow D_1 ; D_2$	$D.type = D_1.type \times D_2.type$
$D \rightarrow \text{fun } id(D):T; B$	$\text{addType}(id.entry, D.type \rightarrow T.type)$
$B \rightarrow \{ S \}$	
$E \rightarrow Id(EList)$ //gọi hàm	$E.type =$ if $\text{lookup}(Id.entry) = t1 \rightarrow t2$ AND $EList.type = t1$ then $t2$ else type_error
$EList \rightarrow E$	$EList.type = E.type$
$EList \rightarrow Elist_1, E$	$EList.type = Elist_1.type \times E.type$

Ví dụ: Kiểm tra kiểu cho chương trình sau

M, N : Integer;

A: Array[10] of Integer;

X: Real;

Fun Foo(X: Integer, Y: Real): Integer;

A: Array[5] of Real;

N : Integer;

{ S }

{

M := Foo(A[10+N], X);

}

Kiểm tra biểu thức kiểu tương đương

//s và t là các biểu thức kiểu

function sequiv(s, t): boolean;

begin

if *s và t có cùng kiểu dữ liệu chuẩn* then return true;

else if s = array(s1, s2) and t = array(t1, t2) then

return sequiv(s1, t1) and sequiv(s2, t2)

else if s = s1 x s2 and t = t1 x t2 then

return sequiv(s1, t1) and sequiv(s2, t2)

else if s = pointer(s1) and t = pointer(t1) then

return sequiv(s1, t1)

else if s = s1 → s2 and t = t1 → t2 then

return sequiv(s1, t1) and sequiv(s2, t2)

else return false;

end;

VAR A, B : MATRIX;

A và B có cùng kiểu?

Chuyển kiểu

float x; int n;

$x + n \rightarrow$ Quy đổi về một kiểu

SẢN XUẤT	QUY TẮC NGŨ NGHĨA
$E \rightarrow \text{num}$	$E.type := int$
$E \rightarrow \text{num.num}$	$E.type := real$
$E \rightarrow \text{id}$	$E.type := \text{lookup}(\text{id.entry})$
$E \rightarrow E_1 \text{ op } E_2$	$E.type :=$ if $E_1.type = int$ and $E_2.type = int$ then int else if $E_1.type = int$ and $E_2.type = real$ then $real$ else if $E_1.type = real$ and $E_2.type = int$ then $real$ else if $E_1.type = real$ and $E_2.type = real$ then $real$ else $type_error$

Chương 4: Phân tích ngữ nghĩa

1. Giới thiệu
2. Bảng ký hiệu
3. Kiểm tra kiểu
4. Chương trình dịch định hướng cú pháp
5. Xử lý sai sót

Giới thiệu

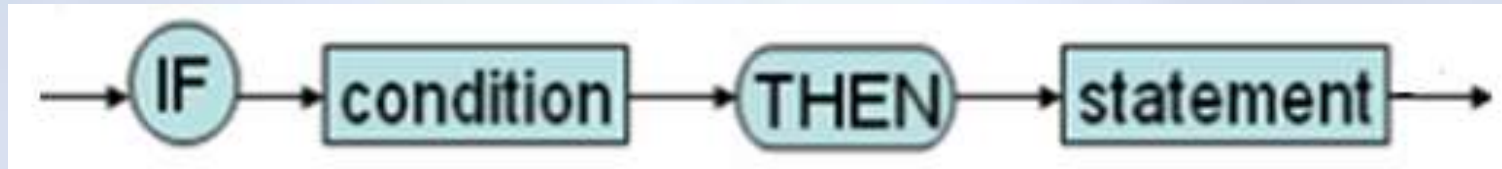
Phân tích cú pháp nếu gặp lỗi

- Gọi tới thủ tục **Error(...)** để báo lỗi
 - Thông báo kiểu của lỗi và vị trí của lỗi
- Sau khi báo lỗi ngừng quá trình dịch lại
 - Bởi văn bản nguồn không khớp với dự đoán → Chương trình dịch không biết đi theo hướng nào
 - VD: gặp statement **(X):=10**; sẽ không biết đi nhánh nào
 - Chương trình dịch có **n** lỗi → cần **n+1** lần dịch
 - Tốn kém
 - Cần điều chỉnh để CTD có thể hồi phục sau khi gặp lỗi

Phương pháp

- Vượt bỏ một đoạn trên văn bản nguồn cho tới khi tìm được một từ tổ cho phép nắm bắt lại được cấu trúc chương trình và có thể tiếp tục làm việc theo dự kiến
- Các từ tổ cho phép nắm bắt được cấu trúc là các **ký hiệu ngừng**. Có thể là
 - Các từ tổ theo sau cấu trúc đang phân tích
 - Khi gặp, cho biết cấu trúc đã bị bỏ qua
 - Là **FOLLOW()** của cấu trúc
 - Các từ tổ trong cấu trúc hiện tại, ít khi bị bỏ sót
 - Được gọi là **Các ký hiệu chốt**
 - Thường là từ khóa của cấu trúc hiện tại

Phương pháp → Ví dụ

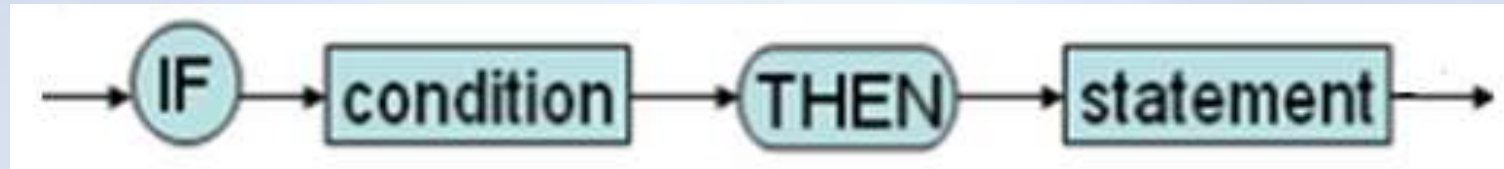


if max > a + * 10 then

↑
Lỗi ở biểu thức (Term) (*thiếu toán hạng hoặc dấu '('*)

- Sau Condition (Expression) là Then, Do, ..
- Khi gặp KW_THEN → đã bỏ qua hết **Condition**
 - Ghi nhận lỗi (coi như đã triển khai xong Condition)
 - Có thể tiếp tục triển khai **Statement**

Phương pháp → Ví dụ



if max > a + 10 Min then

Condition

Lỗi ở Statement, nhánh
if : (thiếu từ khóa **Then**)

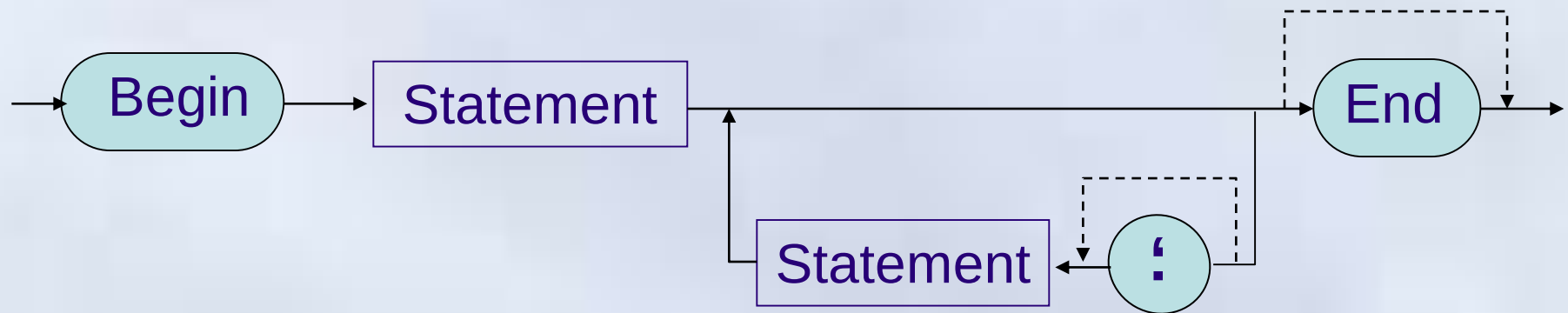
- Sau Statement là **End ; .**
- Không nhất thiết phải bỏ cả **statement** cho tới khi gặp **End ; .**
- Khi đến **Then** là có thể bắt đầu phân tích tiếp

Phương pháp→Thực hiện

- Để thực hiện, cần cho biết các thủ tục phân tích cú pháp (*dùng để triển khai một đích, ví dụ compileBlock(), compileStatement(),...*) có thể hồi phục ở những ký hiệu ngừng nào
 - Các ký hiệu chốt thuộc chính cấu trúc mà thủ tục đang phân tích
 - Các ký hiệu theo sau cấu trúc (phụ thuộc chỗ gọi)
Ví dụ: Đang phân tích Condition, nếu Condition được gọi trong nhánh **IF**, ký hiệu theo sau là **THEN**, còn trong nhánh **WHILE**, ký hiệu theo sau là **DO**
 - Các ký hiệu ngừng được kế thừa từ các cấu trúc (*thủ tục*) bên trên

Phương pháp ghi nhận lỗi

- Khi lỗi có thể đoán biết được không cần vượt quá một đoạn trong chương trình nguồn
 - Ví dụ, lỗi thiếu dấu ; hoặc **End** trong statement
- Có thể sửa lại sơ đồ và viết lại theo sơ đồ mới



Phương pháp ghi nhận lỗi

```

if (token == BEGIN){
    getToken();
    compileStatement();
    while (token  $\in$  [Semicolon]  $\cup$  FIRST(Statement)) do{
        if(Token == Semicolon)
            getToken();
        else
            addError(« Thiếu dấu ';' »);
        compileStatement();
    }
    if (token == END) getToken();
    else addError(« Thiếu từ khóa End »);
}

```

addError() chỉ ghi nhận một thông báo lỗi, không dừng chương trình